GALPyra

Sistema de Gestión y Trazabilidad Avícola

Especificación Completa de Requerimientos del Sistema

Proyecto Integrador III | Universidad Pontificia Bolivariana | Bucaramanga 2025

Zarhet Valentina Fuentes Botia | María Angélica Parra Grazt | Juan Carlos Niño Osorio

Director: Omar Pinzón Ardila

1. REQUERIMIENTOS FUNCIONALES DE INFRAESTRUCTURA

RF-INF-001	Configuración del Repositorio y Control de Versiones
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-INF-002
ENTRADA: Cuentas GitHub del equipo, estructura de ramas definida	**SALIDA:** Repositorio configurado con ramas main, develop y feature con protecciones activas

DESCRIPCIÓN:
Se debe configurar el repositorio principal del proyecto GALPyra en GitHub con estrategia Git Flow. Se crearán las ramas main (producción), develop (integración) y feature/* (desarrollo). Se aplicarán reglas de protección para exigir revisión de código antes de hacer merge.

PRECONDICIONES:
- Cuentas de GitHub de todos los integrantes del equipo activas
- Nombre del repositorio y organización definidos
- Convenciones de commits y nombres de ramas establecidas

POSTCONDICIONES:
- Repositorio creado con estructura de ramas Git Flow
- Branch protection rules activas en main y develop
- Al menos un integrante con permisos de administrador
- README.md con descripción del proyecto creado

FLUJO BÁSICO:
1. Crear repositorio en GitHub bajo la organización del equipo
2. Crear rama develop desde main
3. Configurar reglas de protección: requerir PR y aprobación en main y develop
4. Documentar convención de commits (feat, fix, docs, chore)
5. Crear README.md con descripción del proyecto y estructura
6. Verificar que todos los integrantes tienen acceso al repositorio
7. Si se solicitan cambios antes del merge, se hacen en la misma rama feature

FLUJO ALTERNATIVO:
- Si un integrante no puede acceder, se verifica su vinculación a la organización
- Si se requiere un hotfix urgente, se crea rama hotfix/* desde main con aprobación rápida

RF-INF-002	Configuración del Pipeline CI/CD con GitHub Actions
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-INF-001
ENTRADA: Repositorio configurado, secretos de despliegue del entorno disponibles	**SALIDA:** Pipeline automatizado que ejecuta pruebas y despliega la app al detectar cambios aprobados

DESCRIPCIÓN:
Se debe configurar GitHub Actions para automatizar el ciclo de Integración Continua (CI) y Despliegue Continuo (CD) del proyecto GALPyra. Al hacer push a develop se ejecutan las pruebas; al hacer merge a main se ejecuta el despliegue automático al entorno de producción.

PRECONDICIONES:
- Repositorio con estructura Git Flow configurado (RF-INF-001)
- Entorno de despliegue disponible (nube o CCA)
- Secretos de despliegue almacenados en GitHub Secrets

- Pruebas unitarias implementadas en el proyecto

POSTCONDICIONES:
- Workflow de CI activo: ejecuta pruebas en cada PR
- Workflow de CD activo: despliega automáticamente a main aprobado
- El equipo recibe notificación del resultado del pipeline
- Los despliegues fallidos ejecutan rollback automático

FLUJO BÁSICO:
1. Crear archivo .github/workflows/ci.yml para pruebas automáticas
2. Configurar workflow para ejecutar pruebas en cada PR a develop
3. Crear archivo .github/workflows/cd.yml para despliegue
4. Configurar variables de entorno y secretos en GitHub Secrets
5. Implementar paso de rollback ante fallo de despliegue
6. Verificar que el pipeline funciona correctamente con un commit de prueba

FLUJO ALTERNATIVO:
- Si las pruebas fallan, el pipeline bloquea el merge y notifica al equipo
- Si el despliegue falla, el pipeline revierte al estado anterior

RF-INF-003	**Despliegue del Backend en la Nube (AWS EC2 / CCA UPB)**
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-INF-002
ENTRADA: Instancia de servidor disponible, imagen de la aplicación backend lista	**SALIDA:** Backend de GALPyra desplegado y accesible públicamente via HTTPS en el entorno de producción

DESCRIPCIÓN:
Se debe desplegar el backend de GALPyra (Python/FastAPI) en un servidor cloud (AWS EC2 o CCA de la UPB). El servidor debe ser accesible mediante HTTPS con dominio o IP pública, contar con variables de entorno configuradas y monitoreo básico de disponibilidad.

PRECONDICIONES:
- Instancia de servidor (EC2 o CCA) aprovisionada y con acceso SSH
- Imagen Docker o paquete de la aplicación backend listo
- Certificado SSL disponible o servicio de terminación HTTPS configurado
- Base de datos RDS accesible desde el servidor

POSTCONDICIONES:
- Backend accesible en URL pública con HTTPS
- Endpoint GET /api/health retorna 200 OK
- Variables de entorno de producción configuradas
- Monitoreo básico activo (uptime check)

FLUJO BÁSICO:
1. Conectar a la instancia del servidor vía SSH
2. Instalar Python, pip y dependencias del sistema
3. Clonar el repositorio y configurar variables de entorno de producción
4. Configurar servicio systemd o supervisor para mantener el proceso activo
5. Configurar Nginx como proxy inverso con HTTPS
6. Verificar que GET /api/health retorna 200 OK
7. Configurar firewall para permitir solo puertos 80, 443 y 22

FLUJO ALTERNATIVO:
- Si el servidor no tiene suficiente RAM, se optimiza la configuración de workers de Uvicorn

- Si no se dispone de dominio, se usa la IP pública con certificado autofirmado para desarrollo

RF-INF-004	Configuración del Sistema de Monitoreo y Alertas
PRIORIDAD DE DESARROLLO: Media	**REQUERIMIENTO ASOCIADO:** RF-INF-003
ENTRADA: Servidor backend desplegado y accesible	**SALIDA:** Sistema de monitoreo activo con alertas configuradas ante caídas o anomalías

DESCRIPCIÓN:
Se debe implementar monitoreo básico del backend y la base de datos de GALPyra. Se configurarán alertas por correo o canal de equipo ante caídas del servicio, errores críticos o uso excesivo de recursos.

PRECONDICIONES:
- Backend desplegado y en funcionamiento (RF-INF-003)
- Acceso a herramienta de monitoreo (UptimeRobot, CloudWatch u otra)
- Canal de notificación configurado (correo del equipo)

POSTCONDICIONES:
- Monitoreo de disponibilidad activo con check cada 5 minutos
- Alertas configuradas para notificar caídas en menos de 10 minutos
- Dashboard básico de estado del sistema disponible

FLUJO BÁSICO:
1. Configurar UptimeRobot o AWS CloudWatch para monitorear el endpoint /api/health
2. Definir umbrales de alerta: respuesta > 3 segundos o status != 200
3. Configurar notificaciones al correo del equipo
4. Documentar procedimiento de respuesta ante alertas

FLUJO ALTERNATIVO:
- Si UptimeRobot no está disponible, se usa CloudWatch Basic Monitoring gratuito
- Si el equipo no responde a alertas en 30 minutos, se ejecuta reinicio automático del servicio

RF-INF-005	Configuración de Ambientes de Desarrollo, Pruebas y Producción
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-INF-001
ENTRADA: Repositorio y CI/CD configurados, acceso a servidores disponible	**SALIDA:** Tres ambientes diferenciados: desarrollo local, pruebas (staging) y producción con variables de entorno independientes

DESCRIPCIÓN:
Se deben configurar tres ambientes de ejecución para el proyecto GALPyra: desarrollo (local), pruebas (staging en servidor) y producción. Cada ambiente debe tener sus propias variables de entorno, base de datos y configuración de logging.

PRECONDICIONES:
- Repositorio configurado (RF-INF-001)
- Servidor de staging disponible
- Variables de entorno definidas para cada ambiente

POSTCONDICIONES:
- Ambiente de desarrollo funciona con base de datos local o de prueba
- Ambiente de staging replica la configuración de producción

- Producción usa únicamente variables de entorno seguras, sin credenciales en código

FLUJO BÁSICO:
1. Definir archivo .env.example con todas las variables requeridas
2. Configurar .env.development, .env.staging y .env.production
3. Documentar proceso de configuración del ambiente local para nuevos desarrolladores
4. Asegurar que .env files están en .gitignore
5. Configurar logging diferenciado: verbose en desarrollo, errores críticos en producción

FLUJO ALTERNATIVO:
- Si el servidor de staging no está disponible, se hace QA directamente en producción con rama de prueba
- Si hay conflictos entre variables de entorno, se prioriza la configuración del ambiente activo

RF-INF-006	**Configuración de Variables de Entorno y Secretos**
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-INF-001
ENTRADA: Lista de secretos del sistema: credenciales DB, claves JWT, configuración de servicios externos	**SALIDA:** Variables de entorno configuradas en todos los ambientes sin credenciales expuestas en el repositorio

DESCRIPCIÓN:
Se deben gestionar de forma segura todas las variables de entorno y secretos del proyecto GALPyra. Las credenciales de base de datos, claves JWT y configuraciones sensibles nunca deben incluirse en el código fuente. Se usará GitHub Secrets para CI/CD y archivos .env para entornos locales.

PRECONDICIONES:
- Lista completa de variables de entorno identificada
- Acceso a GitHub Secrets del repositorio
- Archivo .env.example documentado en el repositorio

POSTCONDICIONES:
- Ninguna credencial expuesta en el código o historial de git
- GitHub Secrets configurados para el pipeline CI/CD
- Archivo .env.example actualizado con todas las variables requeridas (sin valores reales)

FLUJO BÁSICO:
1. Identificar todas las variables sensibles del sistema (DB_HOST, DB_PASSWORD, JWT_SECRET, etc.)
2. Crear archivo .env.example con descripciones de cada variable
3. Agregar .env a .gitignore
4. Cargar secretos en GitHub Secrets para uso en workflows
5. Documentar proceso para que nuevos integrantes configuren su .env local

FLUJO ALTERNATIVO:
- Si se detecta una credencial expuesta en el historial de git, se invalida inmediatamente y se genera una nueva
- Si GitHub Secrets no está disponible, se usan variables de entorno del servidor directamente

2. REQUERIMIENTOS FUNCIONALES DE BASE DE DATOS

RF-BD-001	Diseño e Implementación del Modelo de Datos
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BD-002
ENTRADA: Modelo entidad-relación definido con entidades del dominio avícola	**SALIDA:** Esquema de base de datos MySQL creado con todas las tablas, relaciones, índices y restricciones definidas

DESCRIPCIÓN:
Se debe diseñar e implementar el modelo de datos del sistema GALPyra en MySQL (AWS RDS). El modelo debe incluir las entidades: usuarios, galpones, lotes, registros de mortalidad, registros de producción de huevos, eventos sanitarios, y registros de alimentación. Todas las relaciones deben estar definidas con claves foráneas.

PRECONDICIONES:
- Requisitos funcionales del sistema definidos
- Instancia MySQL en AWS RDS disponible y accesible
- Convenciones de nomenclatura de tablas y columnas establecidas

POSTCONDICIONES:
- Todas las tablas creadas con sus columnas, tipos de datos y restricciones
- Claves primarias y foráneas implementadas
- Diagrama Entidad-Relación (DER) documentado y actualizado
- Script SQL de creación del esquema versionado en el repositorio

FLUJO BÁSICO:
1. Diseñar el DER con las entidades identificadas del dominio avícola
2. Definir tipos de datos apropiados para cada campo (fechas, cantidades, textos)
3. Establecer convenciones de nomenclatura (snake_case para columnas)
4. Implementar el esquema mediante script SQL o migraciones SQLAlchemy/Alembic
5. Crear claves primarias auto-incrementales y foráneas con ON DELETE apropiado
6. Documentar el DER en el repositorio del proyecto

FLUJO ALTERNATIVO:
- Si hay cambios en el modelo durante el desarrollo, se gestiona mediante migraciones (RF-BD-004)
- Si una entidad no cabe en el modelo inicial, se agrega mediante migración sin eliminar datos existentes

RF-BD-002	Configuración y Despliegue de MySQL en AWS RDS
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BD-001
ENTRADA: Cuenta AWS con permisos para crear instancias RDS, VPC y grupos de seguridad configurados	**SALIDA:** Instancia MySQL en AWS RDS activa, accesible desde el backend y con backups automáticos configurados

DESCRIPCIÓN:
Se debe aprovisionar y configurar la instancia de base de datos MySQL en Amazon RDS para el proyecto GALPyra. La instancia debe estar en la misma región que el servidor backend, con el grupo de seguridad configurado para permitir acceso únicamente desde el servidor de aplicación.

PRECONDICIONES:
- Cuenta AWS activa con permisos IAM para gestionar RDS
- VPC y subnets disponibles en la región seleccionada

- Grupo de seguridad configurado para restringir acceso

POSTCONDICIONES:
- Instancia RDS MySQL activa y accesible desde el backend
- Backup automático configurado con retención de 7 días
- Usuario de base de datos con permisos mínimos necesarios creado
- Cadena de conexión configurada en las variables de entorno del backend

FLUJO BÁSICO:
1. Crear instancia RDS MySQL db.t3.micro (capa gratuita) en la región seleccionada
2. Configurar grupo de seguridad: permitir puerto 3306 solo desde IP del servidor backend
3. Crear base de datos inicial 'galpyra_db' y usuario con permisos CRUD
4. Habilitar backups automáticos con retención de 7 días
5. Configurar Multi-AZ si el presupuesto lo permite (opcional)
6. Verificar conexión desde el backend con credenciales configuradas

FLUJO ALTERNATIVO:
- Si RDS supera el presupuesto, se evalúa usar MySQL en EC2 o CCA con backups manuales
- Si la conexión falla, se verifica el grupo de seguridad y las reglas de entrada del puerto 3306

RF-BD-003	**Implementación de Índices para Optimización de Consultas**
PRIORIDAD DE DESARROLLO: Media	**REQUERIMIENTO ASOCIADO:** RF-BD-001
ENTRADA: Esquema de base de datos creado con las tablas principales del sistema	**SALIDA:** Índices creados en columnas de búsqueda frecuente para mejorar el tiempo de respuesta de consultas

DESCRIPCIÓN:
Se deben crear índices en las columnas que son frecuentemente usadas en filtros y búsquedas de la aplicación GALPyra, como fechas de registro, ID de galpón y ID de lote. Los índices deben mejorar el rendimiento de las consultas de dashboard y reportes.

PRECONDICIONES:
- Esquema de base de datos implementado (RF-BD-001)
- Consultas frecuentes del sistema identificadas

POSTCONDICIONES:
- Índices creados en columnas de fecha, galpón_id y lote_id en tablas de registros
- Tiempo de respuesta de consultas de reporte < 2 segundos con datos de prueba
- Índices documentados en el DER

FLUJO BÁSICO:
1. Identificar las consultas más frecuentes del dashboard y reportes
2. Analizar columnas usadas en cláusulas WHERE y ORDER BY
3. Crear índices compuestos donde aplique (fecha + galpón_id)
4. Verificar mejora en tiempo de respuesta con EXPLAIN de MySQL
5. Documentar los índices creados en el DER

FLUJO ALTERNATIVO:
- Si un índice no mejora el rendimiento, se elimina para evitar overhead de escritura
- Si las consultas siguen lentas, se considera particionamiento de tablas por fecha

RF-BD-004	Implementación de Migraciones de Base de Datos con Alembic
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BD-001
ENTRADA: Proyecto backend Python configurado, conexión a base de datos establecida	**SALIDA:** Sistema de migraciones configurado que permite evolucionar el esquema de BD de forma controlada y versionada

DESCRIPCIÓN:
Se debe configurar Alembic como herramienta de migraciones de base de datos para el proyecto GALPyra (backend Python/FastAPI). Cada cambio en el modelo de datos debe implementarse como una migración versionada, permitiendo avanzar (upgrade) o revertir (downgrade) cambios de forma controlada.

PRECONDICIONES:
- Proyecto backend Python/FastAPI configurado (RF-BACK-001)
- Conexión a MySQL en AWS RDS establecida (RF-BD-002)
- SQLAlchemy configurado como ORM

POSTCONDICIONES:
- Alembic configurado e integrado con SQLAlchemy
- Migración inicial que crea el esquema completo disponible
- Comandos de upgrade y downgrade funcionando correctamente
- Migraciones versionadas en el repositorio bajo /migrations

FLUJO BÁSICO:
1. Instalar Alembic: pip install alembic
2. Inicializar Alembic en el proyecto: alembic init migrations
3. Configurar alembic.ini con la cadena de conexión a MySQL
4. Crear migración inicial con el esquema completo
5. Ejecutar alembic upgrade head para aplicar el esquema
6. Verificar que las tablas fueron creadas correctamente en RDS
7. Documentar comandos de migración en el README del proyecto

FLUJO ALTERNATIVO:
- Si hay conflictos entre migraciones, se resuelven manualmente editando el archivo de migración
- Si la migración falla en producción, se ejecuta alembic downgrade para revertir

RF-BD-005	Procedimientos de Backup y Recuperación de Datos
PRIORIDAD DE DESARROLLO: Media	**REQUERIMIENTO ASOCIADO:** RF-BD-002
ENTRADA: Instancia RDS activa con datos de producción	**SALIDA:** Política de backup implementada con snapshots automáticos y procedimiento de restauración documentado

DESCRIPCIÓN:
Se debe establecer e implementar la política de backup de la base de datos de GALPyra. AWS RDS ofrece backups automáticos que deben estar configurados. Adicionalmente, se deben hacer snapshots manuales antes de cada despliegue mayor y documentar el procedimiento de restauración.

PRECONDICIONES:
- Instancia RDS en producción activa (RF-BD-002)
- Permisos IAM para gestionar snapshots de RDS

POSTCONDICIONES:
- Backups automáticos de RDS configurados con retención de 7 días
- Al menos un snapshot manual de la BD inicial disponible

- Procedimiento de restauración documentado y probado
- Alerta configurada ante fallo en backup automático

FLUJO BÁSICO:

1. Verificar que los backups automáticos de RDS están habilitados con retención de 7 días
2. Crear snapshot manual antes de cada despliegue mayor
3. Probar el proceso de restauración desde un snapshot en ambiente de pruebas
4. Documentar paso a paso el proceso de restauración
5. Configurar alerta en CloudWatch ante fallo en backup

FLUJO ALTERNATIVO:

- Si el almacenamiento de snapshots supera el límite gratuito, se reducen los días de retención a 3
- Si la restauración falla, se usa el snapshot más reciente disponible como punto de recuperación

RF-BD-006	Implementación de Tabla de Auditoría y Logs del Sistema
PRIORIDAD DE DESARROLLO: Baja	**REQUERIMIENTO ASOCIADO:** RF-BD-001
ENTRADA: Esquema de base de datos implementado, acciones del sistema identificadas	**SALIDA:** Tabla de auditoría creada que registra las operaciones críticas realizadas sobre los datos del sistema

DESCRIPCIÓN:

Se debe implementar una tabla de auditoría en la base de datos de GALPyra que registre las operaciones de creación, modificación y eliminación de datos críticos (lotes, eventos sanitarios, mortalidad). Cada registro debe incluir el usuario que realizó la acción, la fecha y hora, y el tipo de operación.

PRECONDICIONES:

- Esquema de base de datos implementado (RF-BD-001)
- Sistema de autenticación funcional para identificar al usuario

POSTCONDICIONES:

- Tabla 'audit_log' creada con campos: id, tabla, operacion, usuario_id, fecha_hora, datos_anteriores, datos_nuevos
- Operaciones CRUD sobre tablas críticas generan registros de auditoría
- Los logs de auditoría no pueden ser eliminados por usuarios normales

FLUJO BÁSICO:

1. Crear tabla audit_log con los campos definidos
2. Implementar función/trigger que registre cambios en tablas críticas
3. Asegurar que solo el administrador puede consultar los logs de auditoría
4. Verificar que los registros de auditoría se crean correctamente con datos de prueba

FLUJO ALTERNATIVO:

- Si los triggers de BD generan problemas de rendimiento, se implementa auditoría a nivel de aplicación en el backend
- Si el volumen de logs crece demasiado, se implementa purga automática de registros con más de 6 meses

3. REQUERIMIENTOS FUNCIONALES DE BACKEND (API REST)

RF-BACK-001	Configuración del Proyecto Backend y Arquitectura Base
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BD-001
ENTRADA: Requerimientos funcionales del sistema, motor de base de datos MySQL seleccionado	**SALIDA:** Proyecto backend Python/FastAPI configurado con arquitectura en capas, dependencias instaladas y estructura definida

DESCRIPCIÓN:
Se debe configurar el proyecto backend del sistema GALPyra usando Python con FastAPI. Se adoptará una arquitectura en capas (Routers, Services, Repositories, Models). Se instalarán las dependencias necesarias y se configurará la conexión a MySQL usando SQLAlchemy como ORM.

PRECONDICIONES:
- Repositorio configurado (RF-INF-001)
- Python 3.10+ instalado en el entorno de desarrollo
- Conexión a MySQL en AWS RDS disponible

POSTCONDICIONES:
- Proyecto FastAPI iniciado con estructura de carpetas por capas
- Conexión a MySQL probada y funcionando con SQLAlchemy
- Variables de entorno configuradas (RF-INF-006)
- Servidor arranca y endpoint GET /api/health responde 200 OK

FLUJO BÁSICO:
1. Crear entorno virtual Python: python -m venv venv
2. Instalar dependencias: fastapi, uvicorn, sqlalchemy, pymysql, alembic, python-jose, passlib, pydantic
3. Crear estructura: /routers, /services, /repositories, /models, /schemas, /middleware
4. Configurar conexión a MySQL en database.py con SQLAlchemy
5. Crear endpoint de health check GET /api/health
6. Verificar que el servidor arranca con uvicorn main:app --reload

FLUJO ALTERNATIVO:
- Si hay incompatibilidad entre versiones de dependencias, se fija la versión estable en requirements.txt
- Si MySQL no está disponible, se usa SQLite para desarrollo local

RF-BACK-002	Módulo de Autenticación y Gestión de Sesiones JWT
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BACK-001
ENTRADA: Proyecto backend configurado, tabla de usuarios en la base de datos	**SALIDA:** Endpoints de autenticación funcionales: login, refresh token y logout con tokens JWT

DESCRIPCIÓN:
Se debe implementar el módulo de autenticación de GALPyra usando JSON Web Tokens (JWT). El sistema debe emitir access tokens con expiración de 60 minutos y refresh tokens con expiración de 7 días. Las contraseñas se almacenarán hasheadas con bcrypt.

PRECONDICIONES:
- Proyecto backend configurado (RF-BACK-001)
- Tabla usuarios creada en la base de datos (RF-BD-001)
- Librería python-jose y passlib instaladas

POSTCONDICIONES:
- POST /api/auth/login retorna access_token y refresh_token con credenciales válidas
- POST /api/auth/refresh renueva el access_token con refresh_token válido
- POST /api/auth/logout invalida el token activo
- Las contraseñas se almacenan con hash bcrypt en la base de datos

FLUJO BÁSICO:
1. Configurar SECRET_KEY y algoritmo HS256 para JWT
2. Implementar función de hash de contraseñas con passlib/bcrypt
3. Crear endpoint POST /api/auth/login: validar credenciales, emitir JWT
4. Implementar middleware de autenticación para proteger rutas
5. Crear endpoint POST /api/auth/refresh para renovar tokens
6. Implementar POST /api/auth/logout con invalidación de token
7. Probar con colección de Postman todos los flujos de autenticación

FLUJO ALTERNATIVO:
- Si el token expira durante una operación, el sistema retorna 401 y el cliente debe refrescar
- Si se detectan múltiples intentos fallidos (5+), se bloquea el usuario por 15 minutos

RF-BACK-003	API REST - Módulo de Gestión de Galpones y Lotes
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BACK-002
ENTRADA: Autenticación JWT implementada, tablas de galpones y lotes en la BD	**SALIDA:** CRUD completo para galpones y lotes con validaciones y respuestas JSON estandarizadas

DESCRIPCIÓN:
Se deben implementar los endpoints REST para la gestión de galpones y lotes de aves. Los galpones representan las instalaciones físicas del productor y los lotes son los grupos de aves asignados a cada galpón. Todas las operaciones requieren autenticación JWT.

PRECONDICIONES:
- Autenticación JWT funcional (RF-BACK-002)
- Tablas galpones y lotes creadas en MySQL (RF-BD-001)

POSTCONDICIONES:
- GET /api/galpones retorna lista de galpones del usuario autenticado
- POST /api/galpones crea un nuevo galpón con nombre, ubicación y capacidad
- GET /api/galpones/{id}/lotes retorna lotes del galpón
- POST /api/lotes crea un nuevo lote con raza, fecha_ingreso y cantidad_inicial
- Validaciones de campos requeridos retornan error 422 con descripción

FLUJO BÁSICO:
1. Crear modelos SQLAlchemy: Galpon y Lote
2. Crear schemas Pydantic para validación de entrada y salida
3. Implementar repositorios con operaciones CRUD para galpones y lotes
4. Crear servicios con lógica de negocio (validar capacidad, estado del lote)
5. Implementar routers con los endpoints definidos
6. Agregar middleware de autenticación a todas las rutas
7. Probar todos los endpoints con Postman

FLUJO ALTERNATIVO:
- Si se intenta registrar más aves de las que caben en el galpón, se retorna error 400 con mensaje descriptivo
- Si el galpón tiene lotes activos, no se puede eliminar; se retorna error 409

RF-BACK-004	API REST - Módulo de Control de Mortalidad
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BACK-003
ENTRADA: Módulo de galpones y lotes implementado, tabla de mortalidad en la BD	**SALIDA:** Endpoints para registro y consulta de mortalidad por lote con cálculo automático de aves activas

DESCRIPCIÓN:
Se deben implementar los endpoints para registrar y consultar eventos de mortalidad de aves por lote. Cada registro debe incluir fecha, cantidad de bajas, causa y observaciones. El sistema debe actualizar automáticamente el conteo de aves activas del lote al registrar mortalidad.

PRECONDICIONES:
- Módulo de lotes funcional (RF-BACK-003)
- Tabla registros_mortalidad creada en MySQL

POSTCONDICIONES:
- POST /api/lotes/{id}/mortalidad registra bajas y actualiza aves_activas del lote
- GET /api/lotes/{id}/mortalidad retorna historial de mortalidad con filtros de fecha
- El campo aves_activas del lote se actualiza correctamente tras cada registro
- Tasa de mortalidad acumulada calculada y disponible en el endpoint

FLUJO BÁSICO:
1. Crear modelo SQLAlchemy: RegistroMortalidad
2. Implementar lógica de actualización de aves_activas en el servicio
3. Crear endpoint POST /api/lotes/{id}/mortalidad con validaciones
4. Implementar GET con filtros de fecha (fecha_inicio, fecha_fin)
5. Calcular y retornar tasa de mortalidad en la respuesta
6. Agregar campo observaciones opcional al registro

FLUJO ALTERNATIVO:
- Si la cantidad de bajas supera el número de aves activas, se retorna error 400
- Si la causa no está en la lista predefinida, se acepta como 'Otra' con observación obligatoria

RF-BACK-005	API REST - Módulo de Registro de Producción de Huevos
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BACK-003
ENTRADA: Módulo de lotes implementado, tabla de producción en la BD	**SALIDA:** Endpoints para registro diario de producción de huevos con cálculo de indicadores productivos

DESCRIPCIÓN:
Se deben implementar los endpoints para el registro diario de producción de huevos por lote. El sistema debe calcular automáticamente el porcentaje de postura (huevos/aves_activas × 100) y acumular totales por período.

PRECONDICIONES:
- Módulo de lotes funcional (RF-BACK-003)
- Tabla registros_produccion creada en MySQL

POSTCONDICIONES:
- POST /api/lotes/{id}/produccion registra la producción diaria del lote
- GET /api/lotes/{id}/produccion retorna historial con porcentaje de postura calculado
- El sistema calcula automáticamente: porcentaje de postura, promedio semanal y mensual
- Se valida que no exista registro duplicado para la misma fecha y lote

FLUJO BÁSICO:

1. Crear modelo SQLAlchemy: RegistroProduccion
2. Implementar cálculo de porcentaje de postura en el servicio
3. Crear endpoint POST con validación de fecha única por lote
4. Implementar GET con filtros por fecha y cálculo de promedios
5. Agregar lógica de cálculo de tendencia de producción
6. Probar cálculos con datos de prueba

FLUJO ALTERNATIVO:
- Si ya existe un registro para esa fecha y lote, se retorna error 409 con opción de actualizar
- Si las aves_activas del lote es 0, se retorna error 400 indicando que el lote está inactivo

RF-BACK-006	API REST - Módulo de Control Sanitario
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BACK-003
ENTRADA: Módulo de lotes implementado, tabla de eventos sanitarios en la BD	**SALIDA:** Endpoints para registro y consulta del historial sanitario de cada lote con tipos de eventos configurables

DESCRIPCIÓN:
Se deben implementar los endpoints para registrar y consultar el historial sanitario de los lotes de aves. Los eventos sanitarios incluyen: vacunación, tratamiento, diagnóstico y inspección veterinaria. El historial sanitario es clave para la trazabilidad del sistema.

PRECONDICIONES:
- Módulo de lotes funcional (RF-BACK-003)
- Tabla eventos_sanitarios creada en MySQL

POSTCONDICIONES:
- POST /api/lotes/{id}/sanitario registra un evento sanitario con tipo, fecha, descripción y responsable
- GET /api/lotes/{id}/sanitario retorna historial sanitario completo con filtros
- Los tipos de evento están validados contra lista predefinida
- El historial sanitario es accesible para compradores/certificadores con token especial

FLUJO BÁSICO:
1. Crear modelo SQLAlchemy: EventoSanitario
2. Definir tipos de evento: VACUNACION, TRATAMIENTO, DIAGNOSTICO, INSPECCION
3. Implementar endpoints CRUD con autenticación
4. Crear endpoint de consulta pública con token de trazabilidad (RF-BACK-009)
5. Agregar campo responsable (veterinario o técnico) al registro

FLUJO ALTERNATIVO:
- Si el tipo de evento no está en la lista predefinida, se rechaza con error 422
- Si se requiere acceso al historial sin autenticación completa, se usa el token de trazabilidad

RF-BACK-007	API REST - Módulo de Registro de Alimentación
PRIORIDAD DE DESARROLLO: Media	**REQUERIMIENTO ASOCIADO:** RF-BACK-003
ENTRADA: Módulo de lotes implementado, tabla de alimentación en la BD	**SALIDA:** Endpoints para registro y consulta del consumo de alimento por lote con cálculo de conversión alimenticia

DESCRIPCIÓN:

Se deben implementar los endpoints para registrar el consumo de alimento de los lotes de aves. Cada registro debe incluir fecha, tipo de alimento, cantidad en kg y costo. El sistema debe calcular la conversión alimenticia (kg alimento / kg huevo producido).

PRECONDICIONES:
- Módulo de lotes y producción funcionales (RF-BACK-003, RF-BACK-005)
- Tabla registros_alimentacion creada en MySQL

POSTCONDICIONES:
- POST /api/lotes/{id}/alimentacion registra consumo con tipo, cantidad y costo
- GET /api/lotes/{id}/alimentacion retorna historial con consumo acumulado
- Conversión alimenticia calculada y disponible en el reporte del lote

FLUJO BÁSICO:
1. Crear modelo SQLAlchemy: RegistroAlimentacion
2. Implementar cálculo de conversión alimenticia en el servicio
3. Crear endpoints CRUD con validaciones de cantidad y fecha
4. Integrar con módulo de producción para cálculo de eficiencia
5. Agregar campo tipo_alimento con lista predefinida

FLUJO ALTERNATIVO:
- Si no hay registros de producción para el período, se informa que la conversión no puede calcularse
- Si el costo no se registra, el campo es opcional y los reportes económicos quedan incompletos

RF-BACK-008	API REST - Módulo de Dashboard y Métricas
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BACK-003
ENTRADA: Módulos de producción, mortalidad y alimentación implementados	**SALIDA:** Endpoint de dashboard que retorna métricas consolidadas del productor en tiempo real

DESCRIPCIÓN:
Se debe implementar el endpoint de dashboard que consolide las métricas principales del productor: total de aves activas, producción de los últimos 7 días, tasa de mortalidad del período, y alertas activas. Este endpoint debe ser eficiente y responder en menos de 2 segundos.

PRECONDICIONES:
- Módulos de producción, mortalidad y alimentación implementados
- Índices de base de datos optimizados (RF-BD-003)

POSTCONDICIONES:
- GET /api/dashboard retorna métricas consolidadas del productor autenticado
- Tiempo de respuesta < 2 segundos con datos de producción real
- Las métricas incluyen: total aves, producción últimos 7 días, tasa mortalidad, alertas

FLUJO BÁSICO:
1. Diseñar query SQL optimizado para consolidar métricas
2. Implementar endpoint GET /api/dashboard con autenticación JWT
3. Calcular: total aves activas, producción período, tasa mortalidad, alertas de baja producción
4. Agregar caché de 5 minutos para mejorar rendimiento
5. Probar con EXPLAIN de MySQL que los queries usan índices

FLUJO ALTERNATIVO:
- Si los datos de un módulo no están disponibles, se retorna la métrica como null con indicador de falta de datos
- Si el tiempo de respuesta supera 2 segundos, se revisa el query y se agrega caché

RF-BACK-009	API REST - Módulo de Generación de Reportes y Trazabilidad
PRIORIDAD DE DESARROLLO: Media	**REQUERIMIENTO ASOCIADO:** RF-BACK-008
ENTRADA: Datos de producción, mortalidad, sanitario y alimentación disponibles en la BD	**SALIDA:** Endpoints para generación de reportes por período y token de trazabilidad para acceso externo

DESCRIPCIÓN:
Se deben implementar endpoints para generar reportes consolidados por período (semanal, mensual) y un mecanismo de trazabilidad que permita a compradores y certificadores verificar las condiciones operativas del productor mediante un token único de consulta.

PRECONDICIONES:
- Todos los módulos de registro implementados (RF-BACK-004 a RF-BACK-007)
- Tabla de auditoría disponible (RF-BD-006)

POSTCONDICIONES:
- GET /api/reportes genera reporte consolidado por período con filtros
- POST /api/trazabilidad/token genera token único de acceso externo con vigencia configurable
- GET /api/trazabilidad/{token} retorna resumen del productor sin autenticación JWT

FLUJO BÁSICO:
1. Implementar endpoint de reportes con filtros de fecha y tipo
2. Generar tokens únicos de trazabilidad con UUID y fecha de expiración
3. Crear endpoint público de consulta con token de trazabilidad
4. Definir qué información es visible en la consulta pública (resumen productivo y sanitario)
5. Almacenar tokens en la BD con estado activo/expirado

FLUJO ALTERNATIVO:
- Si el token de trazabilidad expira, retorna 401 con mensaje indicando que debe generarse uno nuevo
- Si el reporte abarca más de 90 días, se pagina la respuesta en bloques de 30 días

RF-BACK-010	Módulo de Sincronización Offline - Backend
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BACK-003
ENTRADA: Módulos de registro implementados, mecanismo de sincronización definido en la app móvil	**SALIDA:** Endpoints de sincronización que procesan lotes de registros offline enviados por la app móvil

DESCRIPCIÓN:
Se debe implementar en el backend el soporte para sincronización de datos generados offline por la app móvil Flutter. El backend debe recibir lotes de registros con timestamp local, procesarlos en orden cronológico, detectar duplicados y retornar el resultado de cada operación.

PRECONDICIONES:
- Módulos de registro implementados (RF-BACK-004 a RF-BACK-007)
- Esquema de sincronización definido con el equipo de frontend

POSTCONDICIONES:
- POST /api/sync/upload recibe lote de registros offline y los procesa
- El backend detecta y descarta registros duplicados por ID único local
- La respuesta indica cuáles registros fueron procesados exitosamente y cuáles fallaron
- Los conflictos de datos se resuelven con estrategia last-write-wins por timestamp

FLUJO BÁSICO:
1. Diseñar schema de payload de sincronización: array de registros con id_local, tipo, datos y timestamp
2. Implementar endpoint POST /api/sync/upload con autenticación JWT

3. Procesar registros en orden de timestamp ascendente
4. Detectar duplicados por id_local y descartarlos con status 'already_synced'
5. Retornar respuesta con status individual de cada registro
6. Registrar auditoría de sincronizaciones

FLUJO ALTERNATIVO:
- Si el payload supera 50 registros, se procesa en lotes de 50 y se retorna respuesta parcial
- Si hay conflicto de datos (mismo timestamp, mismo recurso), se aplica last-write-wins con alerta al usuario

4. REQUERIMIENTOS FUNCIONALES DE FRONTEND (APP MÓVIL)

RF-FRONT-001	Configuración del Proyecto de App Móvil Flutter
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BACK-002
ENTRADA: Flutter SDK instalado, URL del backend disponible, Android Studio/VS Code configurado	**SALIDA:** Proyecto Flutter configurado con arquitectura feature-first + clean architecture, dependencias instaladas y build funcional en Android
DESCRIPCIÓN: Se debe configurar el proyecto de la aplicación móvil de GALPyra usando Flutter/Dart. Se adoptará arquitectura feature-first con clean architecture, usando Riverpod para manejo de estado, go_router para navegación, Dio para peticiones HTTP, get_it para inyección de dependencias, y flutter_secure_storage para almacenamiento seguro de tokens.	
PRECONDICIONES: - Flutter SDK 3.x instalado y configurado - Android Studio o VS Code con plugins de Flutter instalados - Dispositivo físico o emulador Android disponible para pruebas	
POSTCONDICIONES: - Proyecto Flutter creado con estructura feature-first - Dependencias instaladas: riverpod, go_router, dio, get_it, flutter_secure_storage - Build en modo debug funcional en emulador Android - Conexión al backend verificada con endpoint /api/health	
FLUJO BÁSICO: 1. Crear proyecto Flutter: flutter create galpyra_app 2. Configurar estructura de carpetas: /features, /core, /shared 3. Agregar dependencias en pubspec.yaml: riverpod, go_router, dio, get_it, flutter_secure_storage 4. Configurar Dio con base URL del backend y interceptores de autenticación 5. Configurar go_router con rutas iniciales 6. Implementar inyección de dependencias con get_it 7. Verificar build: flutter run	
FLUJO ALTERNATIVO: - Si hay conflictos entre versiones de paquetes, se consulta pub.dev para versiones compatibles - Si el emulador no está disponible, se usa un dispositivo físico Android 8.0+ para desarrollo	

RF-FRONT-002	Pantallas de Registro e Inicio de Sesión
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-FRONT-001
ENTRADA: Proyecto Flutter configurado, endpoint de autenticación backend disponible	**SALIDA:** Pantallas de login y registro funcionales con validación de formularios y almacenamiento seguro de tokens
DESCRIPCIÓN: Se deben implementar las pantallas de autenticación de la app GALPyra: pantalla de inicio de sesión (correo + contraseña) y pantalla de registro de nuevo usuario. Los tokens JWT deben almacenarse de forma segura usando flutter_secure_storage. La navegación post-login debe dirigir al dashboard.	
PRECONDICIONES: - Proyecto Flutter configurado (RF-FRONT-001)	

- Endpoint POST /api/auth/login funcional en el backend
- flutter_secure_storage configurado

POSTCONDICIONES:
- Pantalla de login con campos de correo y contraseña funcional
- Validación en tiempo real de campos requeridos y formato de correo
- Token JWT almacenado con flutter_secure_storage tras login exitoso
- Redirección automática al dashboard tras autenticación exitosa
- Mensaje de error descriptivo sin revelar campo incorrecto

FLUJO BÁSICO:
1. Crear feature /features/auth con carpetas data, domain y presentation
2. Implementar AuthRepository con llamada a POST /api/auth/login usando Dio
3. Crear LoginScreen con formulario validado
4. Configurar Riverpod provider para estado de autenticación
5. Almacenar tokens con flutter_secure_storage
6. Configurar go_router para redirigir a dashboard tras login
7. Implementar pantalla de registro con misma arquitectura

FLUJO ALTERNATIVO:
- Si el servidor no está disponible, mostrar mensaje de error de conexión con opción de reintentar
- Si el token expira, redirigir a login automáticamente y mostrar mensaje informativo

RF-FRONT-003	Panel Principal (Dashboard) del Productor
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-FRONT-002
ENTRADA: Autenticación implementada, endpoint de dashboard backend disponible	**SALIDA:** Dashboard con métricas principales del productor visualizadas con gráficas y tarjetas de resumen

DESCRIPCIÓN:
Se debe implementar el panel principal de la app GALPyra que muestra al productor sus métricas más importantes: total de aves activas, producción de los últimos 7 días, tasa de mortalidad del período y alertas activas. El dashboard debe funcionar en modo offline mostrando los últimos datos sincronizados.

PRECONDICIONES:
- Autenticación funcional (RF-FRONT-002)
- Endpoint GET /api/dashboard disponible en el backend
- Base de datos local SQLite/Hive configurada para modo offline

POSTCONDICIONES:
- Dashboard muestra: total aves, producción últimos 7 días, tasa mortalidad y alertas
- Datos se cargan en menos de 3 segundos con conexión disponible
- En modo offline muestra últimos datos sincronizados con indicador de fecha de última sincronización
- Interfaz con íconos y colores intuitivos para productores con baja alfabetización digital

FLUJO BÁSICO:
1. Crear feature /features/dashboard con arquitectura clean
2. Implementar DashboardRepository con llamada a GET /api/dashboard
3. Crear DashboardScreen con tarjetas de métricas
4. Implementar caché local con Hive para modo offline
5. Agregar indicador de última sincronización y estado de conexión
6. Diseñar UI con colores verde/rojo para indicadores de alerta
7. Probar en modo avión para verificar modo offline

FLUJO ALTERNATIVO:
- Si no hay datos en caché local, mostrar pantalla de bienvenida con instrucciones para sincronizar
- Si las métricas están fuera del rango normal, mostrar alerta visual destacada

RF-FRONT-004	Pantallas de Gestión de Galpones y Lotes
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-FRONT-003
ENTRADA: Dashboard implementado, endpoints de galpones y lotes disponibles en backend	**SALIDA:** Pantallas de listado, creación y detalle de galpones y lotes con formularios validados

DESCRIPCIÓN:
Se deben implementar las pantallas para gestionar galpones y lotes de aves. El productor debe poder ver la lista de sus galpones, crear nuevos, ver el detalle de cada galpón con sus lotes activos, y registrar nuevos lotes con información de raza, fecha de ingreso y cantidad inicial.

PRECONDICIONES:
- Dashboard implementado (RF-FRONT-003)
- Endpoints de galpones y lotes funcionales (RF-BACK-003)

POSTCONDICIONES:
- Pantalla de lista de galpones muestra nombre, capacidad y cantidad de aves activas
- Formulario de creación de galpón con validaciones de campos requeridos
- Pantalla de detalle de galpón muestra lotes activos e históricos
- Formulario de registro de lote con raza, fecha_ingreso y cantidad_inicial
- Operaciones funcionan en modo offline con sincronización posterior

FLUJO BÁSICO:
1. Crear feature /features/galpones con arquitectura clean
2. Implementar GalponRepository con CRUD usando Dio y caché Hive
3. Crear pantallas: GalponesList, GalponForm, GalponDetail, LoteForm
4. Implementar navegación con go_router entre pantallas
5. Agregar soporte offline: guardar en Hive si no hay conexión
6. Mostrar indicador de registros pendientes de sincronización

FLUJO ALTERNATIVO:
- Si la cantidad de aves supera la capacidad del galpón, mostrar advertencia antes de guardar
- Si no hay conexión, guardar localmente y mostrar badge de 'pendiente de sincronización'

RF-FRONT-005	Pantallas de Control de Mortalidad
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-FRONT-004
ENTRADA: Pantallas de lotes implementadas, endpoint de mortalidad disponible en backend	**SALIDA:** Pantalla de registro y visualización de mortalidad con historial y gráfica de tendencia

DESCRIPCIÓN:
Se deben implementar las pantallas para registrar y visualizar la mortalidad de aves por lote. El productor debe poder ingresar el número de bajas, la causa y observaciones desde una interfaz simple. El historial debe mostrar la tasa de mortalidad acumulada y una gráfica de tendencia.

PRECONDICIONES:
- Pantallas de lotes implementadas (RF-FRONT-004)
- Endpoint de mortalidad funcional (RF-BACK-004)

POSTCONDICIONES:

- Formulario de registro de mortalidad con campos: cantidad, causa (lista) y observaciones
- Historial de mortalidad con filtros de fecha
- Gráfica de línea mostrando tendencia de mortalidad
- Total de aves activas actualizado visualmente tras cada registro

FLUJO BÁSICO:
1. Crear pantalla MortalidadForm con campos de registro
2. Implementar lista predefinida de causas con dropdown
3. Crear MortalidadHistorial con filtros de fecha
4. Agregar gráfica de línea con fl_chart o charts_flutter
5. Actualizar el contador de aves activas en tiempo real
6. Soporte offline con guardado local

FLUJO ALTERNATIVO:
- Si la cantidad de bajas supera las aves activas, mostrar error con cantidad disponible
- Si la causa es 'Otra', hacer el campo de observaciones obligatorio

RF-FRONT-006	Pantallas de Registro de Producción de Huevos
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-FRONT-004
ENTRADA: Pantallas de lotes implementadas, endpoint de producción disponible en backend	**SALIDA:** Pantalla de registro diario de producción con cálculo de porcentaje de postura y gráfica semanal

DESCRIPCIÓN:
Se deben implementar las pantallas para registrar la producción diaria de huevos por lote. La interfaz debe mostrar el porcentaje de postura calculado automáticamente y una gráfica de producción semanal para que el productor identifique tendencias fácilmente.

PRECONDICIONES:
- Pantallas de lotes implementadas (RF-FRONT-004)
- Endpoint de producción funcional (RF-BACK-005)

POSTCONDICIONES:
- Formulario de registro con campo de cantidad de huevos y fecha
- Porcentaje de postura calculado y mostrado en tiempo real
- Gráfica de barras de producción de los últimos 7 días
- Historial de producción con filtros de fecha
- Validación de registro duplicado por fecha

FLUJO BÁSICO:
1. Crear ProduccionForm con campo de cantidad y selector de fecha
2. Calcular y mostrar porcentaje de postura en tiempo real
3. Implementar gráfica de barras de producción semanal
4. Crear historial con filtros y totales por período
5. Validar fecha única por lote antes de enviar
6. Soporte offline con sincronización pendiente

FLUJO ALTERNATIVO:
- Si ya existe registro para esa fecha, mostrar opción de editar el existente
- Si el porcentaje de postura cae más del 20% respecto al día anterior, mostrar alerta visual

RF-FRONT-007	Pantallas de Control Sanitario
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-FRONT-004
ENTRADA: Pantallas de lotes implementadas, endpoint sanitario disponible en backend	**SALIDA:** Pantallas de registro y consulta del historial sanitario del lote con tipos de eventos categorizados

DESCRIPCIÓN:
Se deben implementar las pantallas para registrar y consultar el historial sanitario de los lotes. El productor debe poder registrar vacunaciones, tratamientos y diagnósticos desde formularios simples. El historial sanitario debe estar disponible para compartir con compradores y certificadores.

PRECONDICIONES:
- Pantallas de lotes implementadas (RF-FRONT-004)
- Endpoint de control sanitario funcional (RF-BACK-006)

POSTCONDICIONES:
- Formulario de registro de evento sanitario con tipo, descripción, responsable y fecha
- Tipos de evento en lista desplegable: Vacunación, Tratamiento, Diagnóstico, Inspección
- Historial sanitario ordenado cronológicamente con filtros por tipo de evento
- Botón para compartir historial sanitario generando token de trazabilidad

FLUJO BÁSICO:
1. Crear SanitarioForm con tipos de evento predefinidos
2. Implementar SanitarioHistorial con filtros por tipo
3. Agregar funcionalidad de compartir historial con token de trazabilidad
4. Soporte offline para registro de eventos
5. Diseñar íconos diferenciados por tipo de evento (jeringa para vacuna, pastilla para tratamiento)

FLUJO ALTERNATIVO:
- Si el responsable no está registrado, permitir ingreso manual de nombre
- Si se selecciona 'compartir historial', informar al usuario qué datos serán visibles externamente

RF-FRONT-008	Pantallas de Seguimiento de Alimentación
PRIORIDAD DE DESARROLLO: Media	**REQUERIMIENTO ASOCIADO:** RF-FRONT-004
ENTRADA: Pantallas de lotes implementadas, endpoint de alimentación disponible en backend	**SALIDA:** Pantalla de registro de consumo de alimento con historial y cálculo de conversión alimenticia

DESCRIPCIÓN:
Se deben implementar las pantallas para registrar el consumo de alimento de los lotes. El productor debe ingresar el tipo de alimento, cantidad en kg y costo opcional. El sistema debe mostrar el historial de consumo y la conversión alimenticia calculada.

PRECONDICIONES:
- Pantallas de lotes implementadas (RF-FRONT-004)
- Endpoint de alimentación funcional (RF-BACK-007)

POSTCONDICIONES:
- Formulario de registro con campos: tipo de alimento, cantidad (kg), costo y fecha
- Historial de consumo con totales acumulados por período
- Conversión alimenticia visible en el detalle del lote
- Soporte offline para registro sin conexión

FLUJO BÁSICO:
1. Crear AlimentacionForm con campos de registro
2. Lista de tipos de alimento predefinida con opción 'Otro'

3. Implementar historial con consumo acumulado
4. Mostrar conversión alimenticia calculada por el backend
5. Campo de costo opcional con validación numérica

FLUJO ALTERNATIVO:
- Si no se ingresa el costo, el campo queda vacío y los reportes económicos mostrarán dato faltante
- Si el tipo de alimento no está en la lista, permitir ingresar texto libre como 'Otro'

RF-FRONT-009	Módulo de Trazabilidad con Código QR
PRIORIDAD DE DESARROLLO: Media	**REQUERIMIENTO ASOCIADO:** RF-FRONT-007
ENTRADA: Historial sanitario y productivo disponibles, endpoint de trazabilidad en backend	**SALIDA:** Funcionalidad de generación de código QR con enlace al perfil público de trazabilidad del productor

DESCRIPCIÓN:
Se debe implementar la funcionalidad de trazabilidad en la app móvil que permita al productor generar un código QR vinculado a su token de trazabilidad. El comprador o certificador puede escanear el QR y acceder al resumen productivo y sanitario del productor sin necesidad de autenticación.

PRECONDICIONES:
- Historial sanitario implementado (RF-FRONT-007)
- Endpoint de trazabilidad funcional (RF-BACK-009)

POSTCONDICIONES:
- Pantalla de trazabilidad muestra QR generado con enlace al perfil público
- El QR puede compartirse por WhatsApp, correo u otras apps
- La vigencia del token de trazabilidad es visible en la pantalla
- El productor puede generar un nuevo QR en cualquier momento (invalida el anterior)

FLUJO BÁSICO:
1. Agregar dependencia qr_flutter para generación de QR
2. Crear TrazabilidadScreen con generación de token desde backend
3. Mostrar QR con el enlace al perfil público
4. Implementar botón de compartir usando share_plus
5. Mostrar fecha de expiración del token activo

FLUJO ALTERNATIVO:
- Si no hay conexión para generar el token, mostrar el QR del token activo anterior si existe
- Si el token expira, notificar al usuario y ofrecer generar uno nuevo

RF-FRONT-010	Pantalla de Reportes y Exportación
PRIORIDAD DE DESARROLLO: Media	**REQUERIMIENTO ASOCIADO:** RF-FRONT-009
ENTRADA: Datos de producción, mortalidad y alimentación disponibles, endpoint de reportes en backend	**SALIDA:** Pantalla de reportes con visualizaciones por período y opción de exportación en PDF

DESCRIPCIÓN:
Se debe implementar la pantalla de reportes de la app GALPyra que permita al productor visualizar resúmenes de producción, mortalidad y alimentación por período (semanal, mensual). El reporte debe poder exportarse en formato PDF para compartir con compradores o entidades certificadoras.

PRECONDICIONES:

- Módulos de producción, mortalidad y alimentación implementados
- Endpoint de reportes funcional (RF-BACK-009)

POSTCONDICIONES:
- Pantalla de reportes con selector de período (semanal, mensual, personalizado)
- Reporte muestra: producción total, mortalidad total, conversión alimenticia y resumen sanitario
- Exportación a PDF funcional y compartible desde la app
- Gráficas de tendencia incluidas en el reporte

FLUJO BÁSICO:
1. Crear ReportesScreen con selector de período y tipo de reporte
2. Mostrar métricas consolidadas del período seleccionado
3. Agregar dependencia pdf/printing para exportación a PDF
4. Implementar generación de PDF con datos del reporte
5. Agregar opción de compartir PDF por WhatsApp o correo

FLUJO ALTERNATIVO:
- Si el período seleccionado no tiene datos suficientes, mostrar mensaje indicando qué datos faltan
- Si la exportación a PDF falla, ofrecer compartir los datos como texto plano

RF-FRONT-011	Implementación del Modo Offline (Sincronización Local-Remoto)
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-FRONT-001
ENTRADA: Proyecto Flutter configurado con Hive, endpoints de sincronización disponibles en backend	**SALIDA:** App funcional en modo offline con cola de sincronización que procesa registros pendientes al recuperar conexión

DESCRIPCIÓN:
Se debe implementar el modo offline completo de la app GALPyra. Todos los registros (mortalidad, producción, sanitario, alimentación) deben poder crearse sin conexión y almacenarse localmente en Hive. Al recuperar conexión, la app debe detectarlo automáticamente y sincronizar los registros pendientes con el backend.

PRECONDICIONES:
- Proyecto Flutter configurado con Hive (RF-FRONT-001)
- Endpoint POST /api/sync/upload funcional en backend (RF-BACK-010)
- Todos los módulos de registro implementados

POSTCONDICIONES:
- La app detecta automáticamente pérdida y recuperación de conexión
- Los registros creados sin conexión se guardan en Hive con estado 'pendiente'
- Al recuperar conexión, sincronización automática de registros pendientes
- Indicador visual en la app del número de registros pendientes de sincronización
- Conflictos de sincronización resueltos con last-write-wins

FLUJO BÁSICO:
1. Configurar connectivity_plus para detectar cambios de conexión
2. Implementar SyncQueue en Hive para almacenar registros pendientes
3. Crear SyncService que detecta conexión y ejecuta sincronización
4. Asignar UUID local a cada registro para deduplicación
5. Mostrar badge con número de registros pendientes en la UI
6. Manejar errores de sincronización y notificar al usuario
7. Probar exhaustivamente escenarios offline: crear registros → desconectar → reconectar → verificar sync

FLUJO ALTERNATIVO:
- Si la sincronización falla parcialmente, reintentar los registros fallidos en el siguiente ciclo

- Si hay conflictos de datos, mostrar notificación al usuario con opción de revisar el conflicto

5. REQUERIMIENTOS NO FUNCIONALES

RNF-001	Usabilidad para Productores con Baja Alfabetización Digital
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-FRONT-002, RF-FRONT-003
ENTRADA: Definición de perfiles de usuario objetivo del sistema	**SALIDA:** Interfaz que puede ser operada por productores rurales con experiencia mínima en tecnología sin necesidad de capacitación extensa
DESCRIPCIÓN: La app móvil de GALPyra debe ser usable por productores avícolas con baja alfabetización digital. La interfaz debe priorizar íconos intuitivos, textos cortos y en español colombiano, flujos de registro en máximo 3 pasos, y elementos visuales de retroalimentación claros.	
PRECONDICIONES: - Definición de flujos de usuario para las funciones principales - Revisión con al menos un productor avícola representativo del público objetivo	
POSTCONDICIONES: - Ningún flujo de registro supera 3 pasos desde el menú principal - Todos los textos están en español colombiano sin tecnicismos innecesarios - La app es utilizable por un productor no capacitado en menos de 15 minutos de exploración	
FLUJO BÁSICO: 1. Diseñar interfaz con íconos representativos para cada módulo (galpón, huevo, jeringa) 2. Limitar formularios a máximo 5 campos por pantalla 3. Usar colores semafóricos (verde/amarillo/rojo) para indicadores de alerta 4. Agregar mensajes de confirmación y error en lenguaje simple 5. Realizar prueba de usabilidad con al menos un productor real	
FLUJO ALTERNATIVO: - Si el productor no puede completar el registro en el tiempo establecido, se simplifica aún más la pantalla - Si los íconos no son reconocidos, se agregan etiquetas de texto corto debajo de cada ícono	

RNF-002	Disponibilidad del Sistema Backend
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-INF-003
ENTRADA: Backend desplegado en servidor de producción con monitoreo activo	**SALIDA:** Sistema backend con disponibilidad mínima del 95% medida mensualmente
DESCRIPCIÓN: El backend de GALPyra debe garantizar una disponibilidad mínima del 95% mensual (equivalente a máximo 36 horas de inactividad al mes). Se excluyen ventanas de mantenimiento programado notificadas con 24 horas de anticipación.	
PRECONDICIONES: - Backend desplegado en servidor de producción (RF-INF-003) - Monitoreo de disponibilidad configurado (RF-INF-004)	
POSTCONDICIONES: - Uptime mensual >= 95% verificable mediante herramienta de monitoreo - Alertas configuradas para notificar al equipo ante caídas mayores a 5 minutos - Procedimiento de recuperación ante caídas documentado y ejecutable en menos de 30 minutos	

FLUJO BÁSICO:
1. Configurar monitoreo de disponibilidad con check cada 5 minutos
2. Documentar procedimiento de reinicio del servicio
3. Configurar servicio systemd con restart automático ante fallos
4. Reportar disponibilidad mensual al equipo y director del proyecto

FLUJO ALTERNATIVO:
- Si el sistema cae fuera del horario del equipo, el reinicio automático debe restaurarlo
- Si la disponibilidad mensual cae por debajo del 95%, se investigan causas y se implementan mejoras

RNF-003	Tiempo de Respuesta de la API REST
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-INF-003, RF-BD-003
ENTRADA: Backend desplegado con índices de base de datos configurados	**SALIDA:** Los endpoints del backend responden en menos de 2 segundos para el 95% de las peticiones bajo carga normal

DESCRIPCIÓN:
El backend de GALPyra debe responder las peticiones en tiempos aceptables para usuarios con conexión móvil limitada. Los endpoints de registro deben responder en menos de 2 segundos y el dashboard en menos de 3 segundos bajo carga normal de hasta 50 usuarios concurrentes.

PRECONDICIONES:
- Backend desplegado (RF-INF-003)
- Índices de base de datos configurados (RF-BD-003)

POSTCONDICIONES:
- Endpoints de registro (POST): respuesta < 2 segundos
- Endpoint de dashboard (GET): respuesta < 3 segundos
- Cumplimiento medido en condiciones de red normal (no modo offline)

FLUJO BÁSICO:
1. Medir tiempos de respuesta de todos los endpoints con Postman
2. Identificar endpoints lentos y optimizar queries SQL
3. Implementar caché en endpoints de dashboard y reportes
4. Verificar que los índices de MySQL están siendo utilizados con EXPLAIN

FLUJO ALTERNATIVO:
- Si un endpoint supera el límite, se optimiza el query antes de pasar a producción
- Si la carga supera 50 usuarios, se evalúa escalar la instancia del servidor

RNF-004	Seguridad y Cumplimiento de la Ley 1581 de 2012 (Habeas Data)
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-BACK-002
ENTRADA: Sistema con manejo de datos personales de productores identificado	**SALIDA:** Sistema con controles de seguridad implementados y tratamiento de datos conforme a la normativa colombiana de protección de datos

DESCRIPCIÓN:
GALPyra debe implementar controles de seguridad para proteger los datos de los productores avícolas conforme a la Ley 1581 de 2012 de Colombia. Las contraseñas deben estar hasheadas, las comunicaciones cifradas con HTTPS y el acceso a datos restringido por rol de usuario.

PRECONDICIONES:
- Sistema de autenticación JWT implementado (RF-BACK-002)
- HTTPS configurado en el servidor de producción (RF-INF-003)

POSTCONDICIONES:
- Todas las contraseñas almacenadas con hash bcrypt (costo >= 10)
- Toda comunicación cliente-servidor usa HTTPS/TLS
- Los datos de un productor no son accesibles por otro productor autenticado
- Política de privacidad incluida en la app describiendo el uso de datos

FLUJO BÁSICO:
1. Verificar que bcrypt está configurado con costo mínimo de 10
2. Validar que todos los endpoints protegidos requieren token JWT válido
3. Implementar validación de propiedad: cada usuario solo accede a sus propios datos
4. Configurar HTTPS en el servidor de producción
5. Redactar y mostrar política de privacidad en la app

FLUJO ALTERNATIVO:
- Si se detecta acceso no autorizado a datos ajenos, se bloquea la cuenta y se notifica al administrador
- Si el certificado SSL expira, el pipeline de CI/CD debe alertar 30 días antes

RNF-005	**Operación en Modo Offline con Sincronización Diferida**
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-FRONT-011, RF-BACK-010
ENTRADA: App móvil con Hive configurado, backend con endpoint de sincronización	**SALIDA:** La app opera completamente sin conexión para registro de datos y sincroniza automáticamente al recuperar conectividad

DESCRIPCIÓN:
GALPyra debe funcionar en zonas rurales con conectividad limitada. Todos los módulos de registro deben estar disponibles sin conexión a internet. Los datos deben sincronizarse automáticamente cuando se recupere la conexión, sin pérdida de información.

PRECONDICIONES:
- App Flutter con Hive configurado (RF-FRONT-001)
- Endpoint de sincronización en backend (RF-BACK-010)

POSTCONDICIONES:
- Todos los formularios de registro funcionan sin conexión a internet
- Los datos offline se sincronizan en menos de 60 segundos tras recuperar conexión
- No se produce pérdida de datos en escenarios offline por periodos de hasta 7 días
- El usuario recibe confirmación visual cuando la sincronización es exitosa

FLUJO BÁSICO:
1. Implementar cola de sincronización persistente en Hive
2. Detectar recuperación de conexión con connectivity_plus
3. Ejecutar sincronización automática al recuperar conexión
4. Probar escenario: crear 20 registros offline → conectar → verificar todos sincronizados

FLUJO ALTERNATIVO:
- Si la sincronización falla, los datos permanecen en cola local y se reintenta en el siguiente ciclo
- Si la app lleva más de 7 días sin sincronizar, mostrar advertencia al usuario

RNF-006	Mantenibilidad y Estándares de Código
PRIORIDAD DE DESARROLLO: Media	**REQUERIMIENTO ASOCIADO:** RF-INF-001
ENTRADA: Equipo de desarrollo con convenciones de código definidas	**SALIDA:** Código fuente con cobertura de pruebas >= 60%, documentación de funciones principales y estilo consistente verificado por linter

DESCRIPCIÓN:
El código de GALPyra debe ser mantenible y comprensible para los tres integrantes del equipo. Se deben seguir las convenciones de Python (PEP 8) para el backend y las convenciones de Dart/Flutter para el frontend. Se debe configurar un linter en el CI/CD que rechace código que no cumpla el estilo.

PRECONDICIONES:
- Repositorio configurado (RF-INF-001)
- CI/CD configurado (RF-INF-002)

POSTCONDICIONES:
- Linter configurado: flake8 para Python, flutter analyze para Dart
- El pipeline CI rechaza código que no pase el linter
- Funciones y clases principales documentadas con docstrings/comentarios
- Cobertura de pruebas >= 60% en módulos críticos del backend

FLUJO BÁSICO:
1. Configurar flake8 para el backend Python en el pipeline CI
2. Configurar flutter analyze para la app móvil en el pipeline CI
3. Documentar funciones principales de servicios y repositorios
4. Crear README con instrucciones de configuración del entorno local

FLUJO ALTERNATIVO:
- Si el linter genera muchos falsos positivos, se configuran excepciones justificadas
- Si la cobertura cae por debajo del 60%, el pipeline genera advertencia (no bloquea)

RNF-007	Escalabilidad de la Arquitectura
PRIORIDAD DE DESARROLLO: Baja	**REQUERIMIENTO ASOCIADO:** RF-INF-003
ENTRADA: Arquitectura del sistema definida con servidor backend y base de datos en la nube	**SALIDA:** Arquitectura documentada con capacidad para escalar verticalmente el servidor y la base de datos sin cambios en el código

DESCRIPCIÓN:
La arquitectura de GALPyra debe permitir escalabilidad vertical sin modificaciones al código fuente. El servidor backend debe poder escalar cambiando el tipo de instancia EC2, y la base de datos debe poder escalar cambiando el tipo de instancia RDS.

PRECONDICIONES:
- Backend desplegado en AWS EC2 (RF-INF-003)
- Base de datos en AWS RDS (RF-BD-002)

POSTCONDICIONES:
- El backend puede escalar verticalmente cambiando el tipo de instancia EC2 sin cambios en el código
- La base de datos puede escalar cambiando el tipo de instancia RDS sin migración de datos
- La arquitectura soporta hasta 200 productores concurrentes con instancia t3.small

FLUJO BÁSICO:
1. Documentar la arquitectura actual con capacidades máximas por instancia
2. Documentar el procedimiento de escalado vertical de EC2 y RDS
3. Definir umbrales de carga que justifican el escalado

FLUJO ALTERNATIVO:
- Si se supera el límite de usuarios, se documenta el plan de escalado para el siguiente sprint
- La escalabilidad horizontal queda fuera del alcance del proyecto académico

RNF-008	Compatibilidad con Dispositivos Android
PRIORIDAD DE DESARROLLO: Alta	**REQUERIMIENTO ASOCIADO:** RF-FRONT-001
ENTRADA: Flutter SDK y requisitos mínimos de hardware del usuario objetivo definidos	**SALIDA:** La app funciona correctamente en dispositivos Android 8.0 (API 26) o superior con 2GB RAM mínimo

DESCRIPCIÓN:
La app móvil de GALPyra debe ser compatible con dispositivos Android de gama media-baja, que son los más comunes entre productores rurales en Santander. La versión mínima de Android soportada es 8.0 (Oreo, API nivel 26).

PRECONDICIONES:
- Proyecto Flutter configurado (RF-FRONT-001)
- minSdkVersion definido en build.gradle

POSTCONDICIONES:
- La app se instala y ejecuta correctamente en Android 8.0+
- La app funciona en dispositivos con 2GB RAM sin degradación notable de rendimiento
- El APK resultante pesa menos de 50MB para facilitar la descarga en zonas con datos limitados

FLUJO BÁSICO:
1. Configurar minSdkVersion 26 en build.gradle
2. Probar en emulador Android 8.0 y dispositivo físico de gama media
3. Optimizar el tamaño del APK con tree shaking de Flutter
4. Verificar que las pantallas se adaptan a resoluciones comunes (720p, 1080p)

FLUJO ALTERNATIVO:
- Si se requiere un paquete incompatible con Android 8.0, se busca alternativa compatible
- Si el APK supera 50MB, se aplican técnicas de reducción de tamaño (compresión de assets, deferred components)

RNF-009	Pruebas Unitarias y de Integración
PRIORIDAD DE DESARROLLO: Media	**REQUERIMIENTO ASOCIADO:** RF-BACK-001
ENTRADA: Módulos del backend implementados con lógica de negocio en servicios	**SALIDA:** Suite de pruebas automatizadas con cobertura >= 60% en servicios del backend y pruebas de integración para endpoints críticos

DESCRIPCIÓN:
El backend de GALPyra debe contar con pruebas automatizadas que garanticen el correcto funcionamiento de la lógica de negocio. Se usará pytest para pruebas unitarias de servicios y pruebas de integración de los endpoints REST.

PRECONDICIONES:
- Módulos del backend implementados (RF-BACK-002 a RF-BACK-010)
- pytest instalado en el proyecto

POSTCONDICIONES:
- Pruebas unitarias para servicios: autenticación, mortalidad, producción, sanitario
- Pruebas de integración para endpoints críticos del sistema

- Cobertura de pruebas >= 60% en módulos de servicios
- Pruebas ejecutadas automáticamente en el pipeline CI

FLUJO BÁSICO:
1. Instalar pytest y pytest-cov
2. Crear pruebas unitarias para AuthService, MortalidadService, ProduccionService
3. Crear pruebas de integración con TestClient de FastAPI
4. Configurar reporte de cobertura en el pipeline CI
5. Documentar cómo ejecutar las pruebas localmente

FLUJO ALTERNATIVO:
- Si la cobertura cae por debajo del 60%, el pipeline genera advertencia pero no bloquea el despliegue
- Si una prueba de integración falla de forma intermitente, se marca como flaky y se investiga

RNF-010	Documentación Técnica y de Usuario
PRIORIDAD DE DESARROLLO: Media	**REQUERIMIENTO ASOCIADO:** RF-INF-001
ENTRADA: Sistema completamente desarrollado con todos los módulos funcionales	**SALIDA:** Documentación técnica de la API, manual de despliegue y guía de usuario simple para productores avícolas

DESCRIPCIÓN:
El proyecto GALPyra debe contar con documentación técnica completa: especificación de los endpoints de la API REST (con FastAPI/Swagger automático), diagrama de arquitectura, modelo de datos (DER) y manual de despliegue. Adicionalmente, se debe crear una guía de usuario simple para los productores avícolas.

PRECONDICIONES:
- Sistema completamente desarrollado
- Diagramas de arquitectura y modelo de datos actualizados
- API REST implementada y probada

POSTCONDICIONES:
- Documentación Swagger/OpenAPI disponible en /api/docs del backend desplegado
- Diagrama de arquitectura del sistema actualizado
- Modelo entidad-relación (DER) documentado
- Manual de despliegue paso a paso disponible en el repositorio
- Guía de usuario simple para productores avícolas

FLUJO BÁSICO:
1. Verificar que FastAPI genera automáticamente la documentación Swagger en /api/docs
2. Crear o actualizar el diagrama de arquitectura del sistema
3. Exportar el DER desde la herramienta de modelado usada
4. Crear manual de despliegue con pasos detallados en el README
5. Crear guía de usuario final con lenguaje simple e ilustraciones
6. Publicar toda la documentación en el repositorio del proyecto

FLUJO ALTERNATIVO:
- Si la documentación de la API es extensa, Swagger la genera automáticamente desde las anotaciones de FastAPI
- Si el tiempo es limitado, se prioriza la documentación técnica sobre la guía de usuario